

Programación Avanzada

Demostración Formal de Programas.

La demostración formal de programas es la aplicación técnicas, métodos y resultados formales a un programa para demostrar que funciona correctamente (es un programa correcto).

¿ En que sentido un programa es correcto ?

Si se tiene un conjunto de datos de entrada P para un programa C , entonces si al ejecutar el programa, se obtiene un conjunto de datos Q y que es el esperado, entonces el programa es correcto.

Hace lo que tiene que hacer el programa.

En este sentido de programa correcto, no se toma en cuenta:

- ✚ Tiempo de ejecución.
- ✚ Recursos utilizados.
- ✚ Código eficiente.

Programación Avanzada

Uno de los métodos formales de demostración de programas correctos son las Ternas de Hoare.

Charles Antony Richard Hoare. Nació el 11 de Enero de 1934 en Colombo Sri Lanka. Inventó el método de ordenación QuickSort en 1960. Premio Turing 1980. Actualmente trabaja en Microsoft.



Programación Avanzada

Este método consiste en hacer un proceso de inferencia lógica para cada tipo de instrucción ejecutable de un programa. Por lo tanto para cada tipo de instrucción ejecutable existe una regla de inferencia.

Cada regla de inferencia para ser aplicada, asume un conjunto previo de condiciones que se cumplen, llamadas precondiciones.

Después de aplicar la regla de inferencia, mediante el proceso de inferencia lógica, se genera otro conjunto de condiciones, llamado poscondiciones.

Por ello a este método también se le conoce como precondiciones y poscondiciones de Hoare.

Las precondiciones y poscondiciones establecen valores de variables.

Programación Avanzada

Las precondiciones serían como los valores iniciales de las variables en un programa.

Las precondiciones y poscondiciones también se les conoce como aserciones o aseveraciones.

Ejemplos de precondiciones y poscondiciones:

✚ $\{ x = 3, y \leq z, z > 4 \}$

✚ $\{ \}$ Aseveración vacía, no compromete nada.

✚ $\{ x^2 \neq 3 \}$

✚ $\{ a = b, i \geq j \}$

Las precondiciones se van a denotar como $\{P\}$ y las poscondiciones como $\{Q\}$, donde P y Q son un conjunto de relaciones entre variables y constantes.

Programación Avanzada

Si C es una parte del código, entonces cualquier aserción $\{P\}$ se denomina precondition de C si $\{P\}$ sólo implica al estado inicial. Cualquier aserción $\{Q\}$ se denomina postcondición si $\{Q\}$ sólo implica el estado final, que es la aseveración después de ejecutar el código C .

Por código o parte de un código entendemos desde una sentencia (instrucción) o conjunto de sentencias hasta un programa completo.

La concatenación de dos códigos $C1$ y $C2$ supone la ejecución secuencial de dichos códigos y vendrá representada de la siguiente forma: $C1 ; C2$.

Un estado de un código es un conjunto de valores que toma en ese instante el conjunto de variables que conforman la estructura de datos del código.

Programación Avanzada

Las precondiciones establecen las condiciones que deben satisfacer los datos de entrada para que el programa pueda realizar lo que debe de hacer.

Las postcondiciones establecen las condiciones de salida que son aceptadas como soluciones correctas al ejecutar el programa.

- ✚ Ejemplo: Programa que calcula la raíz cuadrada de un número.
 - ✚ Precondición: Que el número no sea negativo.
 - ✚ Postcondición: La raíz cuadrada del número.

Programación Avanzada

Un programa es una secuencia de instrucciones que transforman un estado inicial, en un estado final.

El estado inicial es el estado anterior a la ejecución de un código.

El estado final es el estado posterior a la ejecución de dicho código.

Un estado E satisface a una aseveración P , cuando el valor de las variables en el estado E se cumple en P .

Ejemplo.

✚ El estado $E = \{ x = 2 \}$ satisface la aseveración $P = \{ x \geq 2 \}$ porque los valores de las variables en E , $x = 2$ se cumple en $x \geq 2$.

✚ El estado $E = \{ x \geq 2 \}$ no satisface la aseveración $P = \{ x = 2 \}$ porque los valores de las variables en E , por ejemplo $x = 5$ no se cumple en $x = 2$, porque $5 \neq 2$.

Programación Avanzada

Una terna de Hoare está formada por los siguientes elementos:

$$\{P\} C \{Q\}$$

Algunos ejemplos:

✚ $\{y \neq 0\} x := 1/y \{x = 1/y\}$

✚ A veces puede no existir la precondition:

$$\{\} a := b \{a = b\}$$

✚ Esto significa que siempre se obtienen estados que satisfacen la postcondición independientemente de la precondition. Es decir, no importan los valores establecidos en la precondition.

✚ $\{\}$ representa la aserción vacía que se puede entender como “verdadera para todos los estados”. Es decir, satisface cualquier aseveración.

Programación Avanzada

Algunas propiedades de la aseveración vacía:

Si P es una aseveración cualquiera, entonces:

$$\vdash \{ \{ \} \wedge P \} = P$$

$$\vdash \{ \{ \} \vee P \} = \{ \}$$

$$\vdash \{ k = k \} = \{ \}$$

$$\vdash \{ C / 2 + 1 = C / 2 + 1 \} = \{ \}$$

$$\vdash \{ (a+b) = (a+b) \} = \{ \}$$

Programación Avanzada

Si $\{P\} C \{Q\}$ es un código con la precondition $\{P\}$ y la postcondition $\{Q\}$, entonces $\{P\} C \{Q\}$ es correcto si cada estado inicial posible que satisfaga $\{P\}$ da como resultado un estado final que satisface $\{Q\}$ después de ejecutar el programa C .

Corrección total y parcial.

- ✚ Corrección parcial: El código $\{P\} C \{Q\}$ es parcialmente correcto si el estado final de C , cuando termina el programa (aunque no se le exige esta premisa), satisface $\{Q\}$ siempre que el estado inicial satisface $\{P\}$.
- ✚ Corrección total: Cuando un código además de ser parcialmente correcto, siempre termina.

Programación Avanzada

Los programas sin ciclos siempre terminan, por lo que la corrección parcial implica la corrección total en estos casos. Esta distinción es esencial sólo en el caso de programas que no incluyan ciclos o recursiones.

Reglas de Inferencia.

Si R y S son dos aserciones entonces se dice que R es más fuerte que S si todos los estados que satisfacen a R , también satisfacen a S . O bien, que el valor de las variables en R , se cumplen en S también. La manera de denotar esta relación es:

$$R \Rightarrow S.$$

Si R es más fuerte que S , entonces se dice que S es más débil que R . El símbolo $\Rightarrow$ se le conoce como relación de fortalecimiento.

Programación Avanzada

Por ejemplo:

$\{ i > 1 \}$ es más fuerte que $\{ i > 0 \}$ ya que siempre que $i > 1$ esto implica que $i > 0$. Por lo tanto $\{ i > 1 \} \Rightarrow \{ i > 0 \}$.

Si una aserción R es más fuerte que una aserción S entonces todos los estados que satisfacen R también satisfacen S pero no viceversa.

Un fortalecimiento de la aserción disminuye el número de estados que la pueden satisfacer. *Ver ejemplos anteriores.*

- ✚ Más fuerte $\Rightarrow$ más selectivo, más específico, más restrictivo.
- ✚ Más débil $\Rightarrow$ menos selectivo, más general, menos restrictivo.

La aserción más débil es la vacía $\{ \}$ que considera todos los estados posibles. Es como la constante lógica V (true).

Programación Avanzada

La aserción más fuerte será por tanto F (false), ya que representa que ningún estado satisface esta condición.

Algunas reglas de fortalecimiento.

✚ Si $R \Rightarrow S$ entonces si K es una aseveración cualquiera, entonces:

$$\{ P, K \} \Rightarrow \{ S, K \}$$

Por ejemplo:

$\{ i > 5 \} \Rightarrow \{ i > 0 \}$ y si $K = \{ j > 3 \}$ entonces:

$$\{ i > 5, j > 3 \} \Rightarrow \{ i > 0, j > 3 \}$$

✚ Si $R \Rightarrow S$ y $S \Rightarrow T$ entonces $R \Rightarrow T$

✚ $p \Rightarrow \sim(\sim p)$

Programación Avanzada

Algunas reglas de fortalecimiento.

+ Si A es una aseveración cualquiera, entonces:

$$A \Rightarrow A$$

+ Si A y B son aseveraciones y si se cumple que:

$A \Rightarrow B$ entonces puede suceder que:

$$B \Rightarrow A \text{ o bien que } B \not\Rightarrow A$$

Por ejemplo:

$$\{ y = 4 \} \Rightarrow \{ y \neq 0 \} \text{ pero}$$

$$\{ y \neq 0 \} \not\Rightarrow \{ y = 4 \}$$

Programación Avanzada

Por ejemplo:

$$\{ a > b \} \Rightarrow \sim \{ a \leq b \} \text{ pero}$$
$$\sim \{ a \leq b \} \Rightarrow \{ a > b \}$$

Y de igual manera:

$$\{ a = b \} \Rightarrow \sim \{ a > b \} \wedge \sim \{ a < b \}$$
$$\sim \{ a > b \} \wedge \sim \{ a < b \} \Rightarrow \{ a = b \}$$

✚ Si A, B y C son dos aseveraciones cualesquiera y se cumple que:

$A \Rightarrow B$ entonces se cumple:

$$\{ A, C \} \Rightarrow \{ B, C \}$$

Programación Avanzada

Reglas de inferencia de las Ternas de Hoare

Fortalecimiento de Precondiciones.

Si $\{P\}C\{Q\}$ es correcto y además $P_1 \Rightarrow P$ entonces se puede afirmar que $\{P_1\}C\{Q\}$ es correcto.

Si una parte de un código es correcta bajo la precondición $\{P\}$, entonces permanecerá correcto si se refuerza $\{P\}$.

$$\frac{\begin{array}{c} P_1 \Rightarrow P \\ \{P\}C\{Q\} \end{array}}{\{P_1\}C\{Q\}}$$

Programación Avanzada

Por ejemplo:

Supongamos que la siguiente terna de Hoare es correcta:
 $\{y \neq 0\} \ x := 1/y \ \{x = 1/y\}$, demostrar que también lo es la siguiente:

$$\{y = 4\} \ x := 1/y \ \{x = 1/y\}.$$

$$\frac{\begin{array}{c} y = 4 \Rightarrow y \neq 0 \\ \hline \{y \neq 0\} \ x := 1/y \ \{x = 1/y\} \end{array}}{\{y = 4\} \ x := 1/y \ \{x = 1/y\}}$$

En general:

$$\begin{array}{c} \text{Fortalece} \\ \leftarrow \\ \{P\} \Rightarrow \{Q\} \\ \rightarrow \\ \text{Debilita} \end{array}$$

$P =$  Conjunto pequeño de valores

$Q =$  Conjunto grande de valores

Programación Avanzada

- ✚ La aserción vacía puede ser reforzada para producir cualquier precondition $\{P\}$ sea cual fuere. Es decir:

$$\{ P \} \Rightarrow \{ \}$$

Por ejemplo:

La siguiente terna de Hoare que verdadera:

$$\{ \} a := b \{ a = b \}$$

se puede utilizar para justificar que la terna $\{ P \} a := b \{ a = b \}$ es verdadera, donde $\{ P \}$ puede ser cualquier aseveración.

Dem.

$$\frac{\begin{array}{l} \{ P \} \Rightarrow \{ \} \\ \{ \} a := b \{ a = b \} \end{array}}{\{ P \} a := b \{ a = b \}}$$

Programación Avanzada

Debilitamiento de Postcondiciones.

Si $\{P\}C\{Q\}$ es correcto y si además $Q \Rightarrow Q_1$ es correcto, el principio de debilitamiento de postcondiciones permite concluir que $\{P\}C\{Q_1\}$ es cierto. Esto conduce a la siguiente regla de inferencia:

$$\frac{\begin{array}{c} \{P\}C\{Q\} \\ Q \Rightarrow Q_1 \end{array}}{\{P\}C\{Q_1\}}$$

Por ejemplo:

Si la terna de Hoare $\{\} \text{max} := b \{ \text{max} = b \}$ es correcta, demostrar que es correcta: $\{\} \text{max} := b \{ \text{max} \geq b \}$

$$\frac{\begin{array}{c} \{\} \text{max} := b \{ \text{max} = b \} \\ \text{max} = b \Rightarrow \text{max} \geq b \end{array}}{\{\} \text{max} := b \{ \text{max} \geq b \}}$$

Programación Avanzada

Reforzamiento de la precondition y postcondición de en forma simultánea.

Regla de la conjunción de Precondiciones.

Si C es una parte de un código y se ha establecido que $\{P_1\}C\{Q_1\}$ y $\{P_2\}C\{Q_2\}$ son correctas, se puede afirmar que $\{P_1 \wedge P_2\}C\{Q_1 \wedge Q_2\}$ es correcta.

$$\frac{\begin{array}{c} \{P_1\}C\{Q_1\} \\ \{P_2\}C\{Q_2\} \end{array}}{\{P_1 \wedge P_2\}C\{Q_1 \wedge Q_2\}}$$

Como caso particular se tiene:

$$\frac{\begin{array}{c} \{\}C\{Q_1\} \\ \{P_2\}C\{Q_2\} \end{array}}{\{P_2\}C\{Q_1 \wedge Q_2\}}$$

Programación Avanzada

Demostrar que si las siguientes ternas de Hoare son correctas:

$$\{ \} i := i + 1 \{ i = i + 1 \} \text{ y } \{ i > 0 \} i := i + 1 \{ i > 0 \}$$

Entonces es correcta la siguiente:

$$\{ i > 0 \} i := i + 1 \{ i > 1 \}$$

Aplicando el resultado anterior:

$$\frac{\begin{array}{c} \{ \} i := i + 1 \{ i = i + 1 \} \\ \{ i > 0 \} i := i + 1 \{ i > 0 \} \end{array}}{\{ \} \wedge \{ i > 0 \} i := i + 1 \{ i > 0 \} \wedge \{ i = i + 1 \}}$$

Por otra parte, si $i > 0$ y $i = i + 1$ entonces $i > 1$

$$\therefore \{ i > 0 \} \wedge \{ i = i + 1 \} \Rightarrow \{ i > 1 \}$$

Y aplicando la regla de inferencia de _____ se concluye que:

$$\{ i > 0 \} i := i + 1 \{ i > 1 \} \quad \textit{Ver solución.}$$

Programación Avanzada

Debilitamiento de la precondition y postcondición de en forma simultánea.

Regla de la disyunción de Precondiciones.

Si C es una parte de un código y se ha establecido que $\{P_1\}C\{Q_1\}$ y $\{P_2\}C\{Q_2\}$ son correctas, se puede afirmar que $\{P_1 \vee P_2\}C\{Q_1 \vee Q_2\}$ es correcta.

$$\frac{\begin{array}{c} \{P_1\}C\{Q_1\} \\ \{P_2\}C\{Q_2\} \end{array}}{\{P_1 \vee P_2\}C\{Q_1 \vee Q_2\}}$$

Como caso particular se tiene:

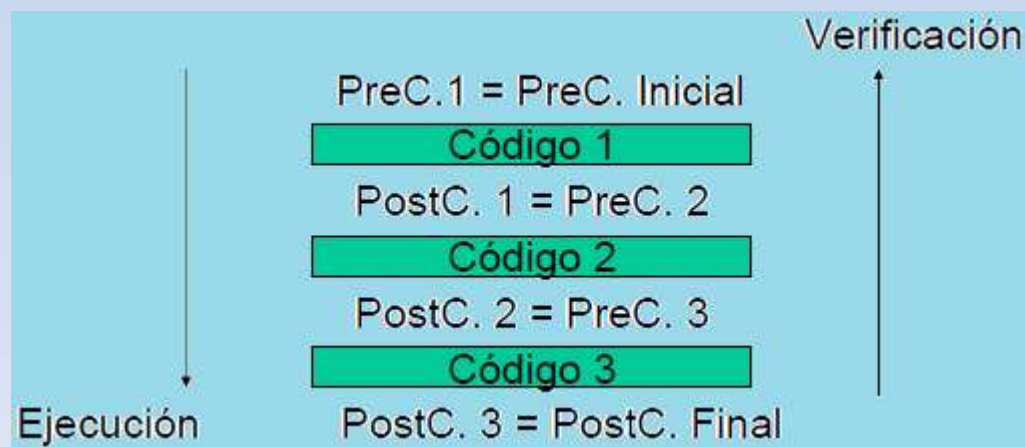
$$\frac{\begin{array}{c} \{\}C\{Q_1\} \\ \{P_2\}C\{Q_2\} \end{array}}{\{P_2\}C\{Q_1 \vee Q_2\}}$$

Programación Avanzada

Demostración de programas correctos.

Para demostrar que un programa es correcto, se parte de la postcondición final del código, es decir de las condiciones que deben satisfacer los resultados correctos al ejecutar el programa. A partir de ahí se deduce la precondition.

El código se verifica en sentido contrario a como se ejecuta.



Programación Avanzada

Si la precondition encontrada es igual a la precondition inicial del programa, el programa es correcto, en otro caso es incorrecto.

Instrucciones de asignación.

Las instrucciones de asignación son de la forma $V := E$, en donde V es una variable y E es una expresión:

$$\{ \} V := E \{ V = E \}$$

Regla de inferencia de la asignación.

Si C es una instrucción de la forma $V := E$ con postcondición $\{Q\}$, entonces la precondition de C puede hallarse sustituyendo todas las ocurrencias de V en Q por E . Si Q_E^V es la expresión que se obtiene de esta manera, entonces para encontrar P en:

Programación Avanzada

$$\{P\} V := E \{Q\}$$

$\{P\} = \{Q_E^V\} = \{V = E, Q\}$ es la expresión que se obtiene sustituyendo V en Q por E .

$$\therefore \{Q_E^V\} V := E \{Q\}$$

La regla de asignación para encontrar una precondition P , puede ser vista intuitivamente de la siguiente manera:

$$\left. \begin{array}{l} a = ? \\ k = 4a \\ k = 12 \end{array} \right\} 4a = 12 \Rightarrow a = 12 / 4 = 3. \text{ O sea, sustituir } k = 4a \text{ en } k = 12 \Rightarrow 4a = 12 \Rightarrow a = 3$$

Es decir: $\{a = ?\} k = 4a \{k = 12\}$ con:

$$P = \{a = ?\} = \{(k = 12)_{4a}^K\} = \{k = 4a, k = 12\} = \{12 = 4a\} = \{a = 3\}$$

$$\therefore \{a = 3\} k = 4a \{k = 12\}$$

Programación Avanzada

Por ejemplo. Determinar la precondition para que la siguiente terna sea correcta:

$$\{ P \} i := 2 i \{ i < 6 \}$$

De acuerdo a la regla de inferencia de asignación:

$$\{ P \} = \{ Q_E^V \} = \{ (i < 6)_{2i}^i \} = \{ i = 2 i, i < 6 \} = \{ 2 i < 6 \} = \{ i < 3 \}$$

$$\therefore \{ i < 3 \} i := 2 i \{ i < 6 \}$$

De forma similar se puede enunciar:

Si C es una instrucción de la forma $V := E$ con la precondition $\{P\}$, entonces la postcondition $\{Q\}$ puede hallarse sustituyendo todas las ocurrencias de la variable que aparece en E por su valor que tiene en $\{P\}$:

$$\{P\} V := E \{Q\}$$

$$\{Q\} = \{ C_{\{P\}}^E \} = \{ E = \{P\}, C \}$$

Programación Avanzada

$$\therefore \{ P \} V := E \{ C_{\{P\}}^E \}$$

donde $C_{\{P\}}^E$ es la expresión que se obtiene sustituyendo E en C por $\{ P \}$.

De igual manera, la regla de asignación de una precondition P puede ser vista intuitivamente de la siguiente manera:

$$\left. \begin{array}{l} a = 3 \\ k = 4a \\ k = ? \end{array} \right\} K = 4 \times 3 \Rightarrow k = 12. \text{ O sea, sustituir } a = 3 \text{ en la instrucción:}$$

$$K = 4a \Rightarrow K = 4 \times 3 = 12$$

Es decir: $\{ a = 3 \} k = 4a \{ k = ? \}$ con:

$$\{ Q \} = \{ C_{\{P\}}^E \} = \{ E = \{ P \}, C \} \text{ es decir:}$$

$$\{ Q \} = \{ (K = 4a)_{\{a=3\}}^{4a} \} = \{ 4a = \{ a = 3 \}, k = 4a \}$$

$$\therefore \{ Q \} = \{ k = 4 \times 3 \} = \{ k = 12 \}$$

La regla dice sustituir en $k = 4a$ el valor de a en $P = \{ a = 3 \}$